المملكة المغربية
ⵜⴰⴳⵍⴷⵉⵜ ⵏ ⵍⵎⵖⵔⵉⴱ
وزارة التربية الوطنية
والتعليم الأولي والرياضة
ⵜⴰⵎⴰⵡⴰⵙⵜ ⵏ ⵓⵙⴳⵎⵉ ⴰⵏⴰⵎⵓⵔ
ⴷ ⴰⵙⵙⵍⵎⴷ ⴰⵎⵣⵡⴰⵔⵓ ⴷ ⵜⵓⵏⵏⵓⵏⵜ

# Examen National du Brevet de Technicien Supérieur (BTS)

**Session 2026**

**- Sujet 3 -**

| Page |
|---|
| 1 / 3 |

| | | |
|---|---|---|
| **Filière :** | **Développement des Applications Informatiques -DAI-** | **Durée :** 2 heures |
| **Épreuve :** | **Développement Mobile Android** | **Coefficient :** 45 |

## CONSIGNES

- Le sujet comporte 7 dossiers.
- Il sera tenu compte de la qualité de rédaction.
- Aucun document n'est autorisé.

**BARÈME DE NOTATION**

| Dossier | Intitulé | Points |
|---|---|---|
| 1 | DAO & SQLite (requêtes complexes, filtres, update, delete) | 10 |
| 2 | Evenements | 6 |
| 3 | SharedPreferences (gestion de session utilisateur) | 6 |
| 4 | Firebase (ajout, lecture, synchronisation, gestion des doublons) | 6 |
| 5 | Cache & Performance | 4 |
| 6 | Gestion de l'état (Bundle, savedInstanceState) | 4 |
| 7 | Analyse & Architecture globale (DAO, SQLite vs Firebase, session) | 4 |
| **Total** | | **40** |

# Contexte général

On souhaite développer une application Android nommée **SmartUserManager** permettant de gérer des utilisateurs avec les fonctionnalités suivantes :

- Gestion locale via SQLite (DAO)
- Synchronisation avec Firebase
- Gestion de session utilisateur
- Mise en cache des données
- Sauvegarde temporaire lors du changement d'état (rotation écran)

Chaque utilisateur possède :

- id (int)
- nom (String)
- age (int)
- email (String)
- actif (boolean)

# Dossier 1 : DAO et SQLite (10 pts)

On suppose que la base de données est déjà créée.

1. Écrire une méthode permettant d'insérer un utilisateur :

   ```
   public long insertUser(User u)
   ```

2. Écrire une méthode permettant de récupérer les utilisateurs dont :
   — l'âge est supérieur à une valeur donnée
   — et l'état actif = true
   (utiliser une requête SQL avec paramètres)

   ```
   public ArrayList<User> getUsersByAgeAndStatus(int ageMin, boolean actif)
   ```

3. Écrire une méthode permettant de rechercher un utilisateur par :
   — nom (LIKE)
   — et email

   ```
   public Cursor rechercherUser(String nom, String email)
   ```

4. Écrire une méthode permettant de modifier un utilisateur en utilisant deux paramètres :
   — id
   — email

   ```
   public int updateUser(int id, String newEmail)
   ```

5. Écrire une méthode permettant de supprimer tous les utilisateurs inactifs.
6. Écrire le code permettant de parcourir un `Cursor` pour remplir une liste d'objets `User`.

   ```
   public ArrayList<User> getUsers()
   ```

7. Expliquer pourquoi il est important de fermer le Cursor.
8. Écrire une méthode permettant de compter le nombre d'utilisateurs retournés.
9. Écrire une méthode permettant de récupérer et parcourir un `Cursor` avec deux paramètres de filtrage :
   — âge minimum
   — état actif (boolean)

   ```
   public ArrayList<User> getUsersByAgeAndStatus(int ageMin, boolean actif)
   ```

# Dossier 2 : Gestion des événements avancés (6 pts)

Dans l'activité `MainActivity`, on dispose des éléments suivants :

```
EditText etNom, etAge;
CheckBox checkActif;
Spinner spinnerType;
TextView tvMessage;
```

1. Écrire le code permettant de gérer le changement d'état de la CheckBox :

   ```
   checkActif.setOnCheckedChangeListener(...)
   ```

   Afficher dans `tvMessage` :
   — "Utilisateur actif" si coché
   — "Utilisateur inactif" sinon
2. Écrire le code permettant de détecter la modification du texte dans `etNom` en utilisant un `TextWatcher` et afficher la longueur du texte dans `tvMessage`.
3. Écrire le code permettant de gérer la perte de focus de `etAge` :

   ```
   etAge.setOnFocusChangeListener(...)
   ```

Si le champ est vide, afficher une erreur.

4. Écrire le code permettant de gérer la sélection d'un élément dans le `Spinner` :

```
spinnerType.setOnItemSelectedListener(...)
```

   Afficher l'élément sélectionné dans `tvMessage`.

5. Expliquer dans un commentaire la différence entre :
   — événement basé sur une action utilisateur directe
   — événement basé sur un changement d'état

# Dossier 3 : SharedPreferences (6 pts)

On souhaite gérer la session utilisateur.

1. Écrire une méthode permettant de sauvegarder :
   — id utilisateur
   — nom
   — état de connexion

```
public void saveSession(User u)
```

2. Écrire une méthode permettant de vérifier si un utilisateur est connecté :

```
public boolean isLoggedIn()
```

3. Écrire une méthode permettant de supprimer la session :

```
public void logout()
```

4. Expliquer dans quel cas SharedPreferences devient insuffisant.

# Dossier 4 : Firebase (6 pts)

On souhaite synchroniser les utilisateurs avec Firebase.

1. Écrire le code permettant d'ajouter une liste d'utilisateurs dans Firebase.
2. Écrire le code permettant de récupérer les utilisateurs depuis Firebase.
3. Expliquer la différence entre :
   — addValueEventListener
   — addListenerForSingleValueEvent
4. On suppose que les utilisateurs sont affichés dans une `ListView` à partir de Firebase.

   Écrire le code permettant de gérer le clic sur un élément de la `ListView` et d'envoyer les informations de l'utilisateur sélectionné vers une nouvelle activité `DetailActivity` via un `Intent`.

   Prototype attendu :

```
listView.setOnItemClickListener(...)
```

   Les données à envoyer :
   — id
   — nom
   — age
   — email
   — actif
5. Dans `DetailActivity`, écrire le code permettant de récupérer les données envoyées dans la méthode :

```
protected void onCreate(Bundle savedInstanceState)
```

6. Dans `DetailActivity`, on dispose d'un formulaire permettant de modifier les informations de l'utilisateur.

   Écrire le code permettant de mettre à jour l'utilisateur dans Firebase lors du clic sur le bouton :

   ```
   btnModifier.setOnClickListener(...)
   ```

7. Expliquer dans un commentaire pourquoi il est important d'utiliser l'identifiant (`id`) pour effectuer la mise à jour dans Firebase.
8. Proposer une amélioration permettant de synchroniser automatiquement les modifications entre Firebase et l'interface utilisateur.

# Dossier 5 : Cache et performance (4 pts)

1. Expliquer le rôle du cache dans une application mobile.
2. Proposer une méthode simple pour mettre en cache une liste d'utilisateurs.
3. Donner un exemple où le cache peut poser problème.

# Dossier 6 : Gestion de l'état et Bundle (4 pts)

1. Expliquer le rôle de `savedInstanceState`.
2. Écrire le code permettant de sauvegarder une liste d'utilisateurs :

   ```
   protected void onSaveInstanceState(Bundle outState)
   ```

3. Écrire le code permettant de restaurer les données dans `onCreate()`.
4. Expliquer la différence entre :
   — sauvegarde temporaire (Bundle)
   — stockage permanent (SQLite / fichier)

# Dossier 7 : Analyse et réflexion (4 pts)

1. Pourquoi utiliser une architecture DAO dans une application Android ?
2. Quels sont les risques d'une mauvaise gestion de la session utilisateur ?
3. Comparer :
   — SQLite
   — Firebase
4. Proposer une architecture complète pour cette application (schéma ou description).